

附件一 项目总结报告模板及要求

《计算机网络》课程项目

总结报告

题目(选题号): _____ 名 称

组 号: _____

任课教师: _____

组 长: _____ 学号 姓名 工作量比例 签名

成 员: _____ 学号 姓名 工作量比例 签名

成 员: _____ 学号 姓名 工作量比例 签名

成 员: _____ 学号 姓名 工作量比例 签名

成 员: _____ 学号 姓名 工作量比例 签名

联系方式: _____ 组长手机号

二〇 年 月 日

基于 Mininet 的校园网络构建与性能分析

一、项目要求

本项目旨在基于 Mininet 网络仿真平台构建一个覆盖多区域的校园网络，实现对真实校园网核心特性的模拟与验证。根据要求，网络需覆盖学生宿舍、教学楼、图书馆、办公楼、财务处及人事处六个功能区域，并在各区域内部实现二层互通、区域之间实现三层互通。所有接入用户需能够访问内网部署的 Web 服务器与 FTP 服务器，完成基本的资源共享功能。针对校园网的数据传输、安全防护要求，提出针对性的策略：

实际校园网中，存在视频流、课件下载、OA 系统、财务系统等不同业务类型。在共享网络带宽资源的条件下，需要通过服务质量（Quality of Service, QoS）机制，优先确保关键业务的充足带宽。针对多个服务器服务本网络拓扑时可能存在的负载不均衡问题，提出基于轮询（Round Robin, RR）的负载均衡策略，尽可能提高所有服务器的整体吞吐量、降低平均处理时延。

网络安全方面，针对校园网中敏感区域保护及外部威胁防御需求，构建由虚拟专用网络（Virtual Private Network, VPN）、访问控制列表（Access Control List, ACL）、Flood 攻击防护、端口扫描检测以及安全事件审计组成的多层安全防护体系。通过 VPN 实现校外用户的受控接入；利用异常流量检测与自动封禁机制抵御端口扫描和 Flood 攻击，并通过安全日志审计实现对网络安全事件的记录与追踪。

本文的主要贡献如下：

（1）构建了一个基于 Mininet 的多区域校园网络仿真平台。该平台覆盖学生宿舍、教学楼、图书馆、办公楼、财务处及人事处六个区域，实现区域内部二层互通和跨区域三层互通，并部署 Web、FTP 等基础网络服务，为校园网络性能分析与安全策略验证提供实验环境。

（2）在基础网络之上设计并实现了服务质量流量调度、基于轮询的负载均衡机制以及由 VPN 远程接入、访问控制列表、异常流量检测和安全事件审计组成的多层安全防护体系，提高了网络资源利用率和系统安全性。

（3）设计并开展了系统性的消融实验与安全验证实验，通过对 QoS、负载均衡和安全防护模块进行逐步剥离与对比，从吞吐量、时延、公平性以及安全事件等多个维度定量评估各功能模块对校园网络性能和安全性的实际贡献。

二、实现原理及方案

Mininet 是目前网络研究与教学中应用最广泛的轻量级网络仿真平台之一^[1]。其通过 Linux 命名空间（namespace）机制为每个虚拟节点创建独立的网络协议栈，利用虚拟以太网对（veth pair）连接各节点，并通过 Open vSwitch 或 Linux Bridge 实现数据平面的转发控制，能够在单台物理主机上构建具有真实 TCP/IP 协议栈行为的大规模网络拓扑。这一特性使 Mininet 在校园网、数据中心网络等场景的仿真验证中得到了较为普遍的采用。

在 QoS 机制方面，现有研究主要围绕链路调度与带宽保障展开，常见方法包括基于队列调度的分层令牌桶（Hierarchical Token Bucket, HTB）、基于优先级的优先级队列（Priority Queuing, PQ）以及基于公平性的加权公平队列（Weighted Fair Queuing, WFQ）。其中，HTB 通过构建层次化 class 结构实现不同业务流的带宽保证与借用机制，适用于多业务并发场景下的分级服务质量控制^[2]；PQ 通过严格优先级调度保证高优先级业务的低时延传输，但可能导致低优先级业务饥饿^[3]；WFQ 则通过按权重分配带宽以实现长期公平性，但在突发流量场景下调度开销较高^[4]。综合考虑实现复杂度与仿真可控性，本项目采用基于 HTB 的分层带宽调度机制作为 QoS 实现方案。

在负载均衡方面，常见调度策略包括轮询（Round Robin, RR）、最少连接数（Least Connections, LC）以及基于响应时间的动态加权调度（Weighted Response Time, WRT）。RR 调度以循环方式依次分配请求至后端服务器，具有无状态、低开销以及易于实现的特点^[5]；LC 策略根据当前活跃连接数动态选择负载最轻的服务器，能够在长连接场景下获得更优性能^[6]；WRT 则进一步引入历史响应时间作为权重因子，以提升系统整体吞吐效率，但实现复杂度较高且依赖精确测量^[7]。考虑到本项目为仿真环境且强调可解释性与对照实验设计，最

终采用 RR 作为基础负载均衡机制。

在网络安全机制研究中,校园网络环境下常见的访问控制与攻击防护技术主要包括基于访问控制列表 (Access Control List, ACL) 的静态规则过滤、基于状态感知的防火墙机制以及基于异常行为分析的入侵检测系统 (Intrusion Detection System, IDS) [8]。其中, ACL 通过预定义源 IP、目的 IP 及端口规则实现对跨区域访问的静态控制,是最基础的访问控制手段,具有实现简单、执行效率高等特点[8]。状态防火墙则通过连接跟踪机制维护会话状态,从而对合法连接流量进行识别与放行,在提升安全性的同时降低误拦截概率 [9]。

IDS 则侧重于对网络流量行为进行分析,其核心方法主要包括基于签名匹配的检测机制、基于统计学习的异常检测方法以及基于时间序列建模的行为分析方法[10][11][12]。其中,基于签名匹配的方法通过预定义攻击特征库对已知攻击进行识别,具有检测精度高但对未知攻击适应性较弱的特点[10];基于统计分析的异常检测方法则通过建立正常流量的统计分布模型,对偏离正常行为的流量进行识别,该类方法在实际实现中常结合滑动时间窗口对短时间内的行为进行聚合分析,从而提高对突发异常流量的检测能力[11]。

在入侵防护方面,针对 ICMP Flood 等高频洪泛攻击,常见的处理方式包括基于速率限制的流量整形机制以及基于深度包检测 (Deep Packet Inspection, DPI) 的入侵防护系统 (Intrusion Prevention System, IPS) [13][14]。其中,基于令牌桶模型的速率限制方法通过限制单位时间内允许通过的报文数量,实现对异常高频流量的直接抑制,该方法通常在内核层实现,具有较低的处理开销与较高的实时性[13];而基于 DPI 的 IPS 机制则通过对报文内容进行深度解析,实现对复杂攻击行为的识别与阻断,但其计算开销较高,通常用于高安全等级网络环境[14]。在实际防护体系中,IDS 与 IPS 构成典型的“检测—响应”联动关系:IDS 负责对异常行为进行识别,而 IPS 则负责对确认的攻击行为实施实时阻断,从而形成完整的安全防护闭环。

不同安全机制在实现复杂度、检测能力与实时性方面各有侧重。ACL 适用于基础访问控制,IDS 适用于异常行为识别,而基于速率限制的 IPS 机制更适用于洪泛类攻击的快速抑制。因此,本项目在校园网络仿真场景中综合采用 ACL 访问控制机制、基于 IDS 与 IPS 组合的端口检测方法以及基于 iptables limit 的 IPS 流量控制机制,并扩展 VPN 身份认证机制,以构建多层次网络安全防护体系。

三、实现过程

3.1 系统的总体结构

在六区域三层路由由拓扑基础上,集成 QoS 流量调度、负载均衡和多层安全防护模块,形成 Final 完整模型。系统总体架构图如图 1 所示:

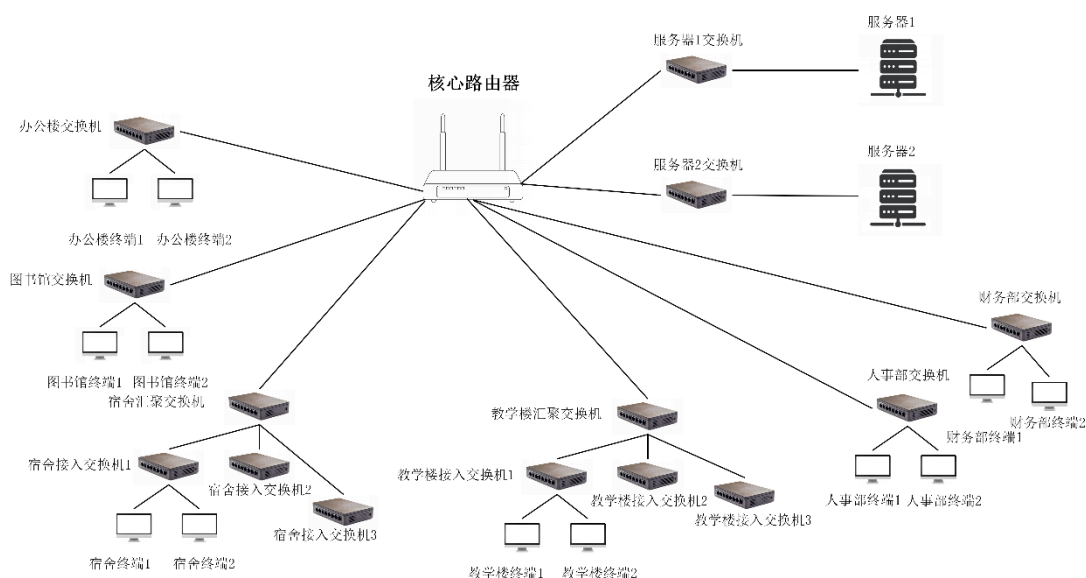


图 1 系统总体架构图

为模拟真实校园网络中用户业务的随机访问特性,系统采用泊松过程 (Poisson Process)

对流量到达行为进行建模。各终端主机按照设定的到达率独立生成业务请求，从而刻画用户请求在时间维度上的随机性与突发性。同时，通过对不同功能区域设置差异化到达率，实现对不同业务负载强度的模拟，以反映校园网络中业务分布不均衡的实际特征。

网络拓扑方面，图书馆、办公楼、财务处和人事处采用单交换机接入结构，宿舍区和教学楼采用接入层-汇聚层的两级交换结构，以模拟高密度终端区域的网络组织方式。两台服务器分别部署于独立服务器子网，并通过独立链路连接核心路由器，为负载均衡实验提供双服务器资源池。

IP 地址规划采用差异化子网设计：宿舍区和教学楼分配较大的 /20 子网，为未来扩容预留空间；图书馆和办公楼分别采用 /23 和 /24 子网；财务处和人事处采用 /26 子网，以满足小规模敏感区域的管理需求；服务器独立部署于 10.0.60.0 地址段，与业务区域完全隔离。该设计兼顾了地址利用率、扩展性以及安全隔离需求，并有利于 ACL 规则配置和路由管理。

所有链路带宽统一设置为 100 Mbps，路由器至各区域链路时延为 5 ms，服务器链路时延为 1 ms。区域内部由交换机完成二层转发，跨区域通信则通过核心路由器实现三层转发。

3.2 基于 HTB 的 QoS 策略

QoS 策略通过分层令牌桶机制在核心路由器的服务器出口链路上实施带宽优先级管理。策略的作用位置是路由器连接两台服务器的出口接口，服务器出口链路的瓶颈带宽设定为 20 Mbps，各区域的下行流量在此处发生竞争，QoS 策略正是在这一竞争点上介入调度。

HTB 采用两级优先级体系，其部署逻辑如下：

Algorithm 1 两级优先级 HTB QoS 部署

Input: E: 出口链路集合, Z: 区域集合 {finance, office, teach, lib, dorm, hr}, B: 链路总带宽, N(z): 区域 z 对应源 IP 子网, R(z): 保证带宽比例, P(z): 调度优先级

Output: HTB 分层带宽调度策略

Process:

```

1: for each e ∈ E do
2: 创建根队列 Q_root(e), 带宽限制为 B
3: for each z ∈ Z do
4: 创建子类 Q(z)
5: Q(z).rate ← B × R(z)
6: Q(z).ceil ← B
7: Q(z).prio ← P(z)
8: 将 Q(z) 绑定至 Q_root(e)
9: 根据 N(z) 将流量映射至 Q(z)
10: 为 Q(z) 配置公平调度策略 (SFQ)
11: end for
12: end for

```

HTB 通过保证带宽 (rate)、最大可用带宽 (ceil) 以及调度优先级 (prio) 共同作用实现链路资源分配。其中，rate 用于定义各业务类别在链路拥塞情况下的最低保障带宽，ceil 用于控制其在链路空闲时的最大可用带宽范围，而 prio 用于决定不同类别在资源竞争时的调度顺序，从而实现基于优先级的差异化带宽分配机制。

在具体参数配置中，财务处业务被设定为高优先级类别，其 rate 设置为 12 Mbps 且 prio 设为 0，以保证其在拥塞场景下优先获得带宽资源；其余业务区域统一设置较低保障带宽 (2 Mbps) 并赋予较低优先级 (prio=7)，以体现非关键业务在资源竞争中的让渡关系。同时，所有类别的 ceil 均设置为 20 Mbps，使各类流量在链路空闲时均可动态借用剩余带宽，从而提高整体链路利用率。该参数配置使关键业务在资源竞争环境中具备更高的调度优先级，同时避免通过绝对隔离方式造成链路资源浪费，从而在保障关键业务性能的同时提升整体网络资源利用效率。

3.3 基于轮询的负载均衡优化

负载均衡模块负责将客户端请求动态分配到两台后端服务器，目标是消除静态绑定方案中因流量分配不均引发的单服务器过载问题，轮询调度器的工作流程如图 2 所示：

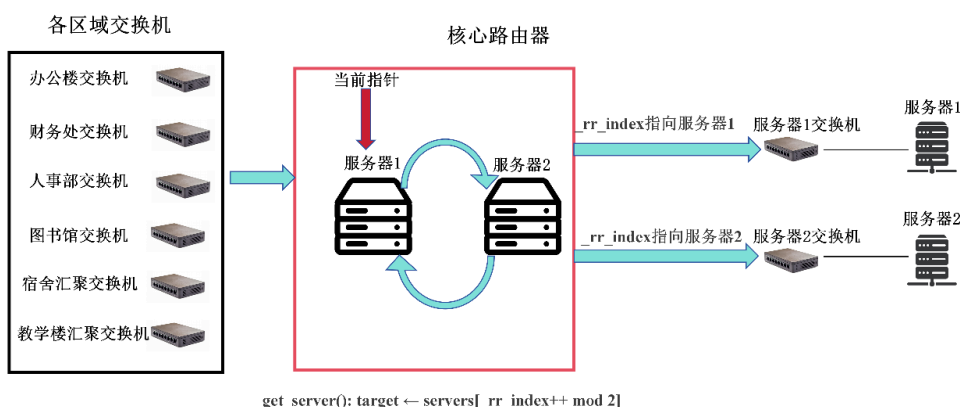


图 2 基于轮询算法的负载均衡优化示意图

调度器内部维护服务器列表 $servers = [S1, S2]$ 与当前轮询指针 rr_index 。为保证多客户端并发访问时的线程安全，系统采用互斥锁保护调度过程中的指针读取、更新与返回操作。每当新的客户端请求到达时，调度器按照轮询顺序选择当前指针对应的服务器，将其返回给调用方，并将指针移动至下一台服务器。当指针到达列表末尾时重新回到起始位置，从而形成循环分配机制。

由于两台服务器拥有相同配置且分别通过独立链路接入核心路由器，轮询策略能够将请求数量较为均匀地分散到两台服务器上，避免部分服务器因请求过度集中而产生排队拥塞，同时减少其他服务器资源闲置的情况。通过提高双服务器资源的整体利用率，系统总吞吐量能够更接近双链路的理论容量上限，并有效降低请求平均响应时间。

3.4 安全策略

通过分层安全防护模型，在核心路由器处构建统一安全控制平面，从访问控制、入侵检测、流量防护及审计记录四个层面实现对校园网络的安全约束与行为监测。

3.4.1 双重访问控制机制

访问控制模块采用分层策略模型实现跨区域与跨网络边界的统一控制，包括校外接入控制与校内区域访问控制两个层级。校外隔离层负责拦截所有来自校园网外部的入站流量。该层以单一主机 IP 地址为匹配条件，在 FORWARD 链中 ICMP Flood 防护规则、状态防火墙规则之后、所有校内 ACL 规则之前插入一条全局 DROP 规则。在系统启动后的默认状态下，该规则持续生效，使校外主机无法与任何校内主机建立连接。

在完成外部流量隔离的基础上，系统进一步在校内网络内部构建区域级访问控制机制，用于实现不同功能区域之间的细粒度权限管理。该机制以源 IP 子网与目的 IP 子网为基本匹配维度，通过规则优先级排序实现访问行为的分级控制：

- (1) 状态防火墙机制允许已建立连接的报文正常回传，以减少对合法会话的额外开销
- (2) 利用白名单策略，允许来自办公楼子网的流量访问财务处与人事处等敏感区域，从而满足业务系统之间的合法通信需求
- (3) 对来自宿舍区、教学区以及图书馆等非授权区域访问财务处与人事处的流量执行拦截与日志记录操作，并将相关事件同步写入系统日志与 SQLite 审计数据库。在所有显式规则之外，默认采用 DROP 策略，对未匹配规则的跨区域流量进行统一拒绝处理。

3.4.2 VPN 接入机制

VPN 模块用于实现校外用户的受控接入。不同于传统 VPN 对加密隧道和专用协议栈的依赖，本系统在 Mininet 仿真环境下采用轻量化抽象模型，通过访问控制策略与身份状态切换机制协同工作，模拟真实 VPN 的接入过程。

在系统启动后的默认状态下，校外主机处于隔离状态，其所有访问请求均受到校外隔离策略限制，无法与校园网内部建立连接。当用户发起 VPN 接入后，通过动态调整访问权限，使校外主机获得受控的校园网访问能力，从而实现远程接入功能。该机制主要包括以下三个阶段：

- (1) 接入认证阶段

系统首先对校外主机的接入请求进行身份确认，在认证完成之前，外部主机始终处于默

认隔离状态，所有进入校园网的流量均被阻断，以保证内部网络边界的安全性。

(2) 身份切换阶段

在认证通过后，系统将主机运行状态由普通外部用户切换为 VPN 用户，并同步更新对应的访问权限，使主机具备受控接入校园网的能力。

(3) 权限授权阶段

在解除外部隔离策略的基础上，为 VPN 用户加载专用访问控制规则，使其能够访问普通业务区域和服务器资源，而对财务处、人事处等敏感区域仍保持访问限制，从而实现基于最小权限原则的远程接入控制。

3.4.3 端口扫描检测与自动封禁

端口扫描的核心检测逻辑在于滑动时间窗口机制。对于每一对“源-目标”IP 组合，模块使用时间戳列表记录该源 IP 在过去 10 秒内访问过的目标端口集合。每当一条新的连接记录到来时，首先清理 10 秒窗口之外的过期记录以维护滑动特性，随后将新的端口与时间戳追加至列表；若超过阈值 20 则判定为端口扫描行为。扫描行为一经判定，立即触发自动封禁：在 FORWARD 链链首同时插入双向 DROP 规则，对被封禁 IP 的所有出入流量实施完全阻断，封禁时长默认为 300 秒，到期后自动清除对应规则。封禁事件与扫描告警事件均被记录至 SQLite 审计数据库。

3.4.4 Flood 攻击防护

ICMP Flood 属于典型的网络层拒绝服务攻击，其主要特征是攻击者通过持续发送大量 ICMP Echo Request 报文，占用链路带宽并增加主机处理负载，从而降低正常业务的可用性。

针对该类高频报文冲击行为，本系统采用基于 iptables limit 模块的速率控制机制，对 ICMP Echo Request 报文实施流量整形与限制。具体实现采用双规则组合策略：第一条规则基于令牌桶机制，对报文通行速率进行控制，其长期速率设定为 1 packet/s，突发容量为 5 packets；第二条规则作为兜底策略，对所有未满足限速条件的 ICMP 报文执行丢弃操作。

由于规则在 netfilter 链中按优先级顺序执行，满足令牌桶条件的正常 ICMP 请求能够通过，而在洪泛攻击场景下超出突发容量的报文将被后续规则直接丢弃，从而实现对异常流量的抑制。

3.4.5 SQLite 安全事件审计

为便于对安全事件进行记录与分析，系统采用 SQLite 数据库保存运行过程中产生的各类安全日志。数据库中的 security_events 表记录事件发生时间、事件类型、源地址、目标地址以及事件描述等信息。

记录内容覆盖访问控制规则触发、端口扫描告警、Flood 攻击检测、IP 自动封禁与解封等安全事件。通过对日志数据进行查询，可以查看特定时间段内的安全事件发生情况，并追踪攻击来源与处理过程，为后续实验结果分析和安全行为验证提供依据。

四、结果分析

4.1 评估指标体系

为量化各策略模块的性能与效果，定义了以下核心评估指标。所有指标均基于实验原始测量数据计算得出。

(1) 吞吐量 (Throughput)

吞吐量表示单位时间内成功传输的数据量，单位为 Mbps。TCP 流的吞吐量由 iperf3 直接统计得到；UDP 流则由发送速率与丢包率共同决定：

$$T_{UDP} = R_{send} \cdot (1 - P_{loss})$$

其中， R_{send} 表示发送速率， P_{loss} 表示丢包率。

(2) 时延 (Latency)

时延表示数据包从发送端到接收端的传输时间，单位为 ms。

TCP 流采用 iperf3 提供的 RTT (Round Trip Time) 作为时延表征：

$$Latency_{TCP} \approx RTT$$

UDP 流采用单向时延平均值：

$$Latency_{UDP} = \frac{1}{N} \sum_{i=1}^N d_i$$

其中, d_i 表示第 i 个数据包单向时延, N 表示数据包总数。

(3) 抖动 (Jitter)

抖动用于衡量时延的波动程度, 定义为相邻数据包时延差的平均值:

$$J = \frac{1}{N-1} \sum_{i=1}^{N-1} |d_{i+1} - d_i|$$

其中, d_i 表示第 i 个数据包时延, N 表示数据包总数。

(4) 丢包率 (Packet Loss Rate)

丢包率定义为丢失数据包占发送总量的比例:

$$P_{loss} = \frac{N_{send} - N_{recv}}{N_{send}} \times 100\%$$

其中, N_{send} : 发送数据包数量, N_{recv} : 成功接收数据包数量。

(5) 利用率 (Utilization)

利用率用于衡量链路资源使用程度:

$$U_i = \frac{T_i}{C_i} \times 100\%$$

其中, T_i 表示第 i 台服务器平均吞吐量, C_i 表示服务器出口链路带宽, 设置为 20 Mbps。

(6) Jain 公平指数 (Jain's Fairness Index)

用于衡量多服务器负载分配均衡性, 其标准定义为:

$$J = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2}$$

其中, x_i 表示第 i 台服务器吞吐量, n 表示服务器数量。 $J = 1$ 时完全均衡, 而 $J = 0.5$ 对应完全不均衡。

4.2 基础模块实验

基础模块测试旨在验证网络拓扑构建、基础通信服务以及 VPN 双重 ACL 接入机制的完整性与正确性。本节将步验证校园网仿真系统的各项基本功能。

4.2.1 网络拓扑构建验证

系统启动后, Mininet 根据 topology.py 中定义的拓扑结构自动创建所有主机、交换机及路由器实例, 并完成链路连接与基础网络初始化。在拓扑构建完成后, 通过 Mininet 内置的 pingall 命令对全网主机连通性进行统一验证, 以检查二层交换与三层路由是否正常工作。

测试覆盖全部网络节点, 包括宿舍区、教学楼、图书馆、办公楼、财务处、人事处以及服务器与外部主机 home_pc, 形成全端到端通信验证, 结果如图 3 所示:

```
[TOPOLOGY] 验证全网连通性...
*** Ping: testing ping reachability
r1 -> dorm1 dorm2 teach1 teach2 lib1 lib2 office1 office2 finance1 finance2 hr1 hr2 server1 server2 home_pc
dorm1 -> r1 dorm2 teach1 teach2 lib1 lib2 office1 office2 finance1 finance2 hr1 hr2 server1 server2 home_pc
dorm2 -> r1 dorm1 teach1 teach2 lib1 lib2 office1 office2 finance1 finance2 hr1 hr2 server1 server2 home_pc
teach1 -> r1 dorm1 dorm2 teach2 lib1 lib2 office1 office2 finance1 finance2 hr1 hr2 server1 server2 home_pc
teach2 -> r1 dorm1 dorm2 teach1 lib1 lib2 office1 office2 finance1 finance2 hr1 hr2 server1 server2 home_pc
lib1 -> r1 dorm1 dorm2 teach1 teach2 lib2 office1 office2 finance1 finance2 hr1 hr2 server1 server2 home_pc
lib2 -> r1 dorm1 dorm2 teach1 teach2 lib1 office1 office2 finance1 finance2 hr1 hr2 server1 server2 home_pc
office1 -> r1 dorm1 dorm2 teach1 teach2 lib1 lib2 office2 finance1 finance2 hr1 hr2 server1 server2 home_pc
office2 -> r1 dorm1 dorm2 teach1 teach2 lib1 lib2 office1 finance1 finance2 hr1 hr2 server1 server2 home_pc
finance1 -> r1 dorm1 dorm2 teach1 teach2 lib1 lib2 office1 office2 finance2 hr1 hr2 server1 server2 home_pc
finance2 -> r1 dorm1 dorm2 teach1 teach2 lib1 lib2 office1 office2 finance1 hr1 hr2 server1 server2 home_pc
hr1 -> r1 dorm1 dorm2 teach1 teach2 lib1 lib2 office1 office2 finance1 finance2 hr2 server1 server2 home_pc
hr2 -> r1 dorm1 dorm2 teach1 teach2 lib1 lib2 office1 office2 finance1 finance2 hr1 server1 server2 home_pc
server1 -> r1 dorm1 dorm2 teach1 teach2 lib1 lib2 office1 office2 finance1 finance2 hr1 hr2 server2 home_pc
server2 -> r1 dorm1 dorm2 teach1 teach2 lib1 lib2 office1 office2 finance1 finance2 hr1 hr2 server1 home_pc
home_pc -> r1 dorm1 dorm2 teach1 teach2 lib1 lib2 office1 office2 finance1 finance2 hr1 hr2 server1 server2
*** Results: 0% dropped (240/240 received)
[TOPOLOGY] pingAll 结果: 0.0% 丢包
```

图 3 全网连通性测试结果

实验结果显示, 全网通信成功率为 100%, 实现了 0%丢包。所有节点均可在预期路径下

完成互联互通，说明交换机转发、路由器转发及子网划分均已正确实现，拓扑结构符合设计要求。

4.2.2 通信与文件传输测试

通信与文件传输测试用于验证校园网内部各区域之间跨子网通信能力以及文件传输机制的正确性。测试在 Mininet CLI 环境下进行，系统通过加载自定义扩展脚本 `network_cli.py` 将通信命令接口注册至运行时环境，从而支持 `send` 命令在仿真拓扑中的调用。

```
mininet> py (exec(open('network_cli.py').read(), globals()), register(net))[1]
[NETWORK_CLI] network_cli.py 已加载
[NETWORK_CLI] 正在初始化...
[RESOURCES] 已检测到 6 个资源文件
[FS] fs_topology/ 已就绪 (16 个节点, 保留 9 个已有文件)
[CLI] 交互式网络通信框架已加载
[CLI] 命令: send <src> <dst> <file> | msg <src> <dst> <msg>
[CLI]          trace <src> <dst> | ls <host>
[CLI]          resources
[CLI] 示例: send dorm1 dorm2 campus.jpg
[CLI] 示例: msg officel dorm1 "hello"
[CLI] 示例: trace dorm1 officel
[NETWORK_CLI] 框架已就绪! 命令: send, msg, trace, ls, resources
[NETWORK_CLI] 文件系统: /home/user/桌面/campus-net/fs_topology/nodes/
[NETWORK_CLI] 传输日志: /home/user/桌面/campus-net/fs_topology/runtime_transfer.json
[INTERACTIVE] 14 个交互函数已就绪
transfer_file, download_http, download_ftp
show_identity, switch_identity, enable_vpn, disable_vpn
show_blacklist, show_conntrack, scan, flood
access_resource, check_perimeter_acl, print_access_result
[NETWORK_CLI] 交互式演示函数已就绪 (enable_vpn / disable_vpn / transfer_file ...)
```

图 4 Mininet CLI 中执行 `send` 命令及加载 `network_cli.py` 的初始化过程

该命令底层基于 netcat 实现数据传输连接建立，并通过 MD5 对传输文件进行一致性校验，以确保数据在跨子网传输过程中的完整性与一致性。

```
mininet> msg officel dorm1 "Meeting at 3pm"
[MSG] officel → dorm1: "Meeting at 3pm"
[ROUTE] 路径: officel → r1 → dorm1
[FS] 文件已落盘: msg_officel_090416_172.txt (76 bytes)
[MSG] 网络连通性已验证 ✓
[MSG] 已写入: fs_topology/nodes/dorm1/files/msg_officel_090416_172.txt
```

图 5 办公楼主机 1 向宿舍区主机 1 成功通信

```
mininet> send officel financel 通知_2026.docx
[SEND] officel → financel: 通知_2026.docx (0.015 MB)
[ROUTE] 路径: officel → s_office → r1 → s_finance → financel
[OK] 传输完成! 耗时: 0.0s, MD5: ✓ 一致, 网络: ✓ 连通
```

图 6 办公楼主机 1 向财务室主机 1 成功传输文件

```
mininet> send server1 dorm1 DMS_essay.zip
[SEND] server1 → dorm1: DMS_essay.zip (2.465 MB)
[ROUTE] 路径: server1 → r1 → dorm1
[OK] 传输完成! 耗时: 0.002s, MD5: ✓ 一致, 网络: ✓ 连通
```

图 7 宿舍区主机 1 从服务器 1 成功下载压缩包文件

```
mininet> send dorm1 financel campus.jpg
[ACL] ACL_DENY (非授权区域禁止访问敏感区域) - 传输被拦截
```

图 8 宿舍区主机 1 向财务处主机 1 传输失败

```
mininet> send dorm1 dorm2 campus.jpg
[SEND] dorm1 → dorm2: campus.jpg (0.251 MB)
[ROUTE] 路径: dorm1 → s_dorm1 → dorm2
[OK] 传输完成! 耗时: 0.0s, MD5: ✓ 一致, 网络: ✓ 连通
```

图 9 宿舍区主机 1 向宿舍区主机 2 成功传输文件

测试结果显示，各普通区域之间均可稳定完成文件传输，且传输结果与源文件 MD5 完

全一致，说明跨子网路由与交换机制工作正常。同时，在白名单规则作用下，办公区至财务处的访问请求能够被正确放行，验证了访问控制策略的正确性。同时，校内 ACL 也正确拦截了校内普通区域向受保护区域的通信。

4.3 QoS 消融实验

为验证 HTB 优先级调度机制对多业务并发场景下网络性能的影响，实验对比完整系统（Final）与对照组（Final-HTB 消融）。完整系统在区域交换机至核心路由器的上行链路部署 HTB 两级优先级策略；对照组关闭 QoS 模块，采用 Linux 默认 pfifo 队列进行数据包转发，其余网络拓扑、业务流量模型、负载均衡机制及安全策略均保持一致。

实验采用泊松过程模拟业务请求的随机到达，各业务流持续时间统一设置为 5 s，总实验时长为 60 s。系统同时运行六路业务流，包括财务处关键业务、教学楼课件下载、办公楼 OA 系统、宿舍区视频流、图书馆网页浏览以及人事处业务流。其中，财务处、教学楼、办公楼、图书馆和人事处采用 TCP 业务模型，宿舍区采用 UDP 视频流模型。为测量时延、抖动和丢包率等实时业务性能指标，在财务处额外部署一路目标带宽为 1 Mbps 的 UDP 探针流，并通过比较其传输质量变化评估 HTB 对关键业务时延稳定性的影响。实验结果如表 1 所示。

表 1 QoS 消融实验结果

区域	协议	吞 吐 量 (Mbps)	时延 (ms)	抖动 (ms)	丢 包 率 (%)
财务处（关键业务-TCP） — Final-HTB 消融	TCP	12.93	18.98	N/A	N/A
财务处（关键业务-TCP） — Final	TCP	15.78	18.70	N/A	N/A
教学楼（课件下载-TCP） — Final-HTB 消融	TCP	11.22	21.46	N/A	N/A
教学楼（课件下载-TCP） — Final	TCP	6.57	20.72	N/A	N/A
办公楼（OA 系统-TCP） — Final-HTB 消融	TCP	8.97	18.98	N/A	N/A
办公楼（OA 系统-TCP） — Final	TCP	8.91	18.63	N/A	N/A
宿舍区（视频流-UDP） — Final-HTB 消融	UDP	12.00	20.87	0.59	0.0
宿舍区（视频流-UDP） — Final	UDP	12.00	20.81	0.85	32.59
图书馆（网页浏览-TCP） — Final-HTB 消融	TCP	11.91	18.89	N/A	N/A
图书馆（网页浏览-TCP） — Final	TCP	10.79	18.55	N/A	N/A
人事处（HR 系统-TCP） — Final-HTB 消融	TCP	10.73	N/A	N/A	N/A
人事处（HR 系统-TCP） — Final	TCP	9.72	N/A	N/A	N/A

区域	协议	吞 吐 量 (Mbps)	时延 (ms)	抖动 (ms)	丢 包 率 (%)
财务处（UDP 探针） Final-HTB 消融	UDP	1.00	18.98	1.76	0.0
财务处（UDP 探针） Final	UDP	1.00	18.70	0.36	0.0

在 Final-HTB 消融组中，所有业务通过 pfifo 队列共享链路资源。财务处关键业务吞吐量达到 12.93 Mbps，已经能够满足其目标带宽需求。启用 HTB 两级优先级策略后，财务处吞吐量进一步提升至 15.78 Mbps，增幅约为 22.0%，说明优先级调度机制能够为关键业务提供更稳定的带宽保障。

而低优先级区域的吞吐量受到不同程度影响。其中，教学楼吞吐量由 11.22 Mbps 降至 6.57 Mbps，降幅达到 41.4%，受影响最为明显；图书馆和人事处吞吐量分别下降至 10.79 Mbps 和 9.72 Mbps，降幅约为 9.4%；办公楼吞吐量变化较小。结果表明，HTB 优先级调度对低优先级业务的影响并非均匀分布，而是主要压缩高需求业务所占用的带宽资源。

宿舍区 UDP 视频流在两组实验中的吞吐量均稳定在 12.00 Mbps，但在 Final 组中出现了 32.59% 的丢包率。这说明在高优先级业务获得优先保障后，低优先级 UDP 流量在链路拥塞时更容易发生队列溢出，从而导致数据包丢失。

财务处 UDP 探针流量在两组实验中均保持 1.00 Mbps 的吞吐量，但其抖动由 1.76 ms 降低至 0.36 ms，下降约 79.5%，说明 HTB 不仅能够提高关键业务的吞吐能力，同时还能够改善其时延稳定性。

综合来看，HTB 两级优先级策略在多业务竞争环境下实现了对财务处关键业务的差异化保障，在提高关键业务吞吐量和降低时延波动的同时，其代价主要表现为部分低优先级业务吞吐量下降以及 UDP 业务丢包率上升，验证了 HTB 在校园网多业务场景下进行差异化服务质量控制的有效性。

4.4 负载均衡消融实验

为评估 Round Robin 负载均衡策略对双服务器系统性能的影响，实验对比完整系统（Final）与对照组（Final-LB）。完整系统采用 Round Robin 轮询机制动态分配请求，对照组关闭负载均衡功能，采用静态映射方式将宿舍区和图书馆请求固定分配至 Server2，其余区域请求固定分配至 Server1。除调度策略外，两组实验的网络拓扑、QoS 配置、安全机制以及业务流量模型均保持一致。

实验采用 HTTP 大文件下载业务模型，请求到达过程服从泊松分布。每次请求下载固定大小为 5 MB 的测试文件，总实验时长为 60 s，两台服务器出口链路带宽均设置为 20 Mbps。通过统计请求完成情况、平均响应时间、服务器吞吐量、链路利用率以及 Jain 公平指数等指标，对比分析动态负载均衡策略与静态映射策略的性能差异。实验结果如表 2 所示。

表 2：负载均衡消融实验结果

调 度 请 求 平 均 总吞吐 S1 S2 S1 利 S2 利 负 载 Jain 指 策略 数 响 应 (Mbps) (Mbps) (Mbps) 用率 用率 方差 数 (s)									
Final-LB	41	11.11	25.25	17.25	8.01	86.2%	40.0%	533.54	0.8820
Final	39	4.36	26.67	13.68	12.99	68.4%	65.0%	2.92	0.9993

注：单链路瓶颈为 20 Mbps，双链路总容量为 40 Mbps，两台服务器的利用率各自独立计算。

从表 2 可以看出，在静态绑定策略下，两台服务器负载分布明显不均。Server1 吞吐量达到 17.25 Mbps，利用率为 86.2%，已接近单链路瓶颈；而 Server2 吞吐量仅为 8.01 Mbps，利用率为 40.0%。对应的负载方差达到 533.54，Jain 公平指数仅为 0.8820，说明系统存在较为严重的负载失衡。其主要原因在于静态映射将办公楼、教学楼、财务处和人事处等主要业

务流量集中于 Server1，导致两台服务器承担的请求压力差异较大。

采用 Round Robin 策略后，两台服务器吞吐量分别为 13.68 Mbps 和 12.99 Mbps，对应利用率为 68.4%和 65.0%，负载分布趋于均衡。负载方差由 533.54 降低至 2.92，Jain 公平指数提高至 0.9993，接近理想公平状态。同时，平均响应时间由 11.11 s 降低至 4.36 s，下降 60.8%，说明轮询调度能够有效缓解单服务器过载，提高系统响应性能。

与静态映射相比，Round Robin 策略使系统总吞吐量由 25.25 Mbps 提高至 26.67 Mbps。虽然仍低于双链路理论容量 40 Mbps，但由于实验业务以 HTTP 大文件下载为主，请求到达过程具有随机性，同时受到 TCP 拥塞控制机制的影响，链路难以始终保持满负载状态。因此，实际吞吐量与理论容量之间存在一定差距。总体来看，Round Robin 策略在响应时延、负载均衡程度以及系统吞吐量等方面均优于静态绑定方案，说明动态请求分配能够有效提高双服务器系统的资源利用效率。

4.5 安全机制实验

安全策略功能验证实验旨在验证系统中访问控制、VPN 接入机制、ICMP/TCP Flood 攻击防护以及端口扫描检测与自动封禁机制的实际运行效果。与前述性能优化类实验不同，本节更侧重于对安全模块行为正确性的验证，即各安全策略是否能够按照设计逻辑正确生效。

4.5.1 双重访问控制与 VPN 接入验证

本实验用于验证校外主机接入控制以及 VPN 状态切换机制的正确性。系统中校外主机 home_pc 在默认状态下处于完全隔离模式，其所有访问请求均被 FORWARD 链中的校外隔离规则阻断；当 VPN 功能被启用后，系统通过状态切换机制为主机分配 VPN_USER 身份，并动态调整 iptables 规则，使其获得受控访问权限。

实验分别测试 VPN 关闭、VPN 开启两种状态下的访问行为，结果如图 10，11，12 所示：

```
mininet> ls home_pc
[home_pc] files/
VPN测试文件.pdf          77323 B MD5:89b8605c
mininet> send home_pc dorm1 VPN测试文件.pdf
[ACL] ACL_EXTERNAL_BLOCK (校外主机未接入VPN, 禁止访问校园网) - 传输被拦截
mininet> send home_pc finance1 VPN测试文件.pdf
[ACL] ACL_EXTERNAL_BLOCK (校外主机未接入VPN, 禁止访问校园网) - 传输被拦截
```

图 10 未开启校园 VPN 时的 home_pc 访问情况

```
mininet> py enable_vpn("home_pc")
[VPN] WARNING: 无法获取 Mininet 网络对象, iptables 规则未修改
[VPN] VPN 已连接 - home_pc 身份: HOME_USER → VPN_USER
[VPN] 虚拟 IP: 10.0.80.10 (池: 10.0.80.0/24)
[VPN] 权限: 可访问校内普通区域, 仍禁止访问财务处/人事处
mininet> send home_pc dorm1 VPN测试文件.pdf
[SEND] home_pc → dorm1: VPN测试文件.pdf (0.074 MB)
[ROUTE] 路径: home_pc → s_home → r1 → sd_agg → s_dorm1 → dorm1
[OK] 传输完成! 耗时: 0.0s, MD5: ✓ 一致, 网络: ✓ 连通
mininet> send home_pc finance1 VPN测试文件.pdf
[ACL] ACL_VPN_DENY (VPN用户禁止访问敏感区域) - 传输被拦截
```

图 11 开启校园 VPN 后的 home_pc 访问情况

```
mininet> py disable_vpn("home_pc")
[VPN] WARNING: 无法获取 Mininet 网络对象, iptables 规则未修改
[VPN] VPN 已断开 - home_pc 身份恢复: HOME_USER
[VPN] 状态: 完全隔离, 无法访问校园网任何主机
mininet> send home_pc dorm1 VPN测试文件.pdf
[ACL] ACL_EXTERNAL_BLOCK (校外主机未接入VPN, 禁止访问校园网) - 传输被拦截
mininet> send home_pc finance1 VPN测试文件.pdf
[ACL] ACL_EXTERNAL_BLOCK (校外主机未接入VPN, 禁止访问校园网) - 传输被拦截
```

图 12 关闭校园 VPN 后的 home_pc 访问情况

在 VPN 关闭状态下, home_pc 无法访问任何校园网资源, 说明校外隔离策略能够有效生效; 在 VPN 开启后, 系统成功解除外部隔离并加载 VPN 专用访问控制规则, 使其能够访问普通区域但仍受敏感区域 ACL 限制; 当 VPN 关闭后系统恢复初始隔离状态, 验证了状态切换机制的可逆性与一致性。

4.5.2 ICMP Flood 攻击防护验证

该实验用于验证系统对 ICMP Flood 攻击的防护能力。ICMP Flood 属于网络层高频探测型攻击, 其核心特征是在短时间内发送大量 ICMP Echo Request 报文, 从而消耗链路带宽及主机处理资源。本实验通过对比启用与未启用防护机制两种状态下的报文通过情况, 评估限速策略的有效性。系统采用 iptables limit 模块实现基于令牌桶的速率控制机制, 其长期速率设置为 1 packet/s, 突发容量设置为 5 packets。实验结果如表 3 所示:

表 3 ICMP Flood 攻击防护效果

攻击类型	状态	总传输报文数	成功报文数
ICMP FLOOD	未启用防护	40	40
	启用防护	40	5

实验结果表明, 在未启用防护机制时, 40 个 ICMP 探测报文全部成功到达目标主机, 成功率为 100%, 说明系统在默认状态下对高频探测流量不存在限制能力。

在启用限速策略后, 成功报文数由 40 下降至 5, 成功率降至 12.5%。该结果与令牌桶机制的参数设定一致: 初始突发容量为 5 packets, 因此攻击开始阶段允许少量报文通过; 随后由于持续发送速率超过长限制 1 packet/s, 后续 35 个报文均被内核直接丢弃。防护机制使 ICMP Flood 有效通过率下降 87.5%, 表明系统能够在无需用户态干预的情况下, 在内核层实现对异常高频流量的稳定抑制, 从而有效保护链路资源与主机处理能力。

4.5.3 端口扫描检测与自动封禁验证

本实验用于验证系统对端口扫描行为的检测能力及自动封禁机制的有效性。检测模块基于用户态 Python 程序实现, 通过维护源 IP 在 10 秒滑动时间窗口内访问的端口数量, 当访问端口数超过阈值 20 时触发扫描告警, 并调用 iptables 在 FORWARD 链首部插入 DROP 规则, 对攻击源进行封禁, 封禁时长为 300 秒。

表 4 端口扫描检测与封禁效果

状态	扫描端口数	成功探测端口数	是否触发告警并封禁
未启用检测	30	30	否
启用检测	30	20	是

未启用检测机制时, 攻击源可以完成全部 30 个端口的连续探测, 系统未对异常访问行为进行限制。而在启用检测机制后, 攻击行为在探测至第 20 个端口时触发滑动窗口阈值条件, 系统立即判定该行为为端口扫描, 并生成安全告警。随后自动执行 iptables 封禁规则, 使攻击源后续 10 个端口探测请求被直接丢弃, 因此最终仅成功探测 20 个端口。

这一结果表明, 基于滑动时间窗口的端口访问统计方法能够有效识别高频多端口探测行为, 而结合 iptables 的动态封禁机制可以在检测触发后立即限制攻击源通信能力, 从而实现了端口扫描行为的检测、告警与自动封禁。

4.5.4 SQLite 安全事件审计验证

SQLite 审计模块用于对系统运行过程中产生的安全事件进行持久化记录。在完成前文进行的 ICMP Flood 攻击防护实验和端口扫描检测实验后, 系统会通过统一事件接口自动将相关信息写入 SQLite 数据库, 并记录事件时间、事件类型、源地址、目标地址、事件详情以及严重级别等内容。在完成上述安全实验后, 对审计数据库进行查询, 得到部分记录如表 5 所示:

表 5 SQLite 安全事件记录

时间	事件类型	源 IP	目标 IP	事件详情	严重级别
2026-06-07 14:29:38	FLOOD	10.0.0.2	10.0.60.2	ICMP Flood 触发, 丢弃报文数: 35	WARNING
2026-06-07 14:29:40	PORT_SCAN	10.0.0.2	10.0.60.2	扫描端口数: 20	CRITICAL

2026-06-07 14:29:40	BAN	10.0.0.2	N/A	封禁原因：端口扫描；封禁时长：300 s	WARNING
------------------------	-----	----------	-----	----------------------	---------

实验结果表明，系统能够对 ICMP Flood 攻击、端口扫描行为以及自动封禁操作进行完整记录。其中，FLOOD 事件记录了异常流量触发及报文丢弃情况；PORT_SCAN 事件记录了扫描行为及触发阈值；BAN 事件则记录了攻击源的封禁原因及持续时间。通过保存攻击源地址、事件类型及处理结果等关键信息，SQLite 审计模块能够为安全事件的追踪与溯源提供数据支持，同时也为系统运行状态分析和后续故障排查提供依据。

五、项目总结

5.1 主要困难与解决方案

项目开发过程中遇到的第一大困难是 Mininet 环境下 QoS 策略作用位置的确定。初始方案将 HTB 部署于各区域上行接口，但实验中不同区域流量之间的竞争效果不显著，原因在于各区域流量最终汇聚于服务器出口链路时才形成真正的瓶颈竞争点。将 HTB 迁移至服务器出口链路后，六个区域的流量在同一个 20 Mbps 瓶颈处发生竞争，HTB 的优先级调度效果才得以清晰呈现。

负载均衡实验中，如何制造可观测的服务器过载条件是当前面临的一个重要问题。在初始拓扑中，两台服务器共享同一上行链路，即便请求被均匀分发，服务器回程流量仍汇聚于同一条链路，轮询策略的效果无从体现。将拓扑修改为双独立上行链路架构后，两台服务器各自拥有独立的 20 Mbps 出口，静态绑定方案中 Server1 负责的区域汇聚流量迅速将其推向饱和，而轮询方案的均衡分发效果通过利用率和负载方差数据得以量化展示。这一架构调整是负载均衡消融实验获得清晰对比结果的关键基础。

安全机制实验的主要挑战在于 ACL 测试方案的选择。初期使用 ping 测试 ACL 黑名单功能，但结果持续出现误判：即便 FORWARD 链的 DROP 规则已正确插入，ICMP 包依然能够穿透。排查后发现，ICMP 报文也受到入侵检测模块中 ICMP Flood 限速规则的影响，两套规则之间发生了干扰。因此在实验设计中对规则执行顺序与状态跟踪策略进行统一约束，通过重置 conntrack 状态并调整 ACL 规则优先级，保证访问控制测试过程中不受状态防火墙机制干扰。

5.2 小组人员分工

姓名	主要负责内容
郑澈	网络拓扑构建、核心路由器配置、基础连通性验证
邵伟轩	基于 HTB 方法的 QoS 策略设计与部署、QoS 消融实验设计与执行
王孟瑞	负载均衡模块实现、负载均衡消融实验设计与执行、安全策略部署
马俊浩	入侵检测模块实现、安全机制实验设计与执行、SQLite 审计系统、报告撰写

5.3 项目小结与展望

本项目基于 Mininet 平台构建了一个覆盖六个业务区域、双服务器架构的校园网络仿真系统，并在基础网络之上集成了 HTB QoS 流量调度、Round Robin 负载均衡以及基于 iptables 的安全防护机制。通过功能验证实验与消融实验，对各模块在带宽保障、负载分发和安全防御等方面的实际效果进行了评估。

QoS 实验结果表明，HTB 两级优先级策略能够在多业务竞争环境下实现差异化服务保障。实验中，财务处关键业务吞吐量由 12.93 Mbps 提升至 15.78 Mbps，增幅达到 22.0%，说明优先级调度机制能够有效提高关键业务的服务质量。同时，资源竞争主要集中于高需求业务，验证了 HTB 在校园网多业务场景下进行精细化资源分配的有效性。

负载均衡实验结果表明，Round Robin 调度能够显著改善双服务器系统的负载分布。与静态绑定策略相比，系统平均响应时间由 11.11 s 降低至 4.36 s，Jain 公平指数由 0.8820 提

升至 0.9993, 说明动态请求分发机制能够有效提高资源利用率并增强系统整体性能。

安全实验结果表明, 系统实现了访问控制、异常流量抑制、端口扫描检测以及安全事件审计等功能。校外主机在默认状态下无法访问校园网资源, 启用 VPN 后则能够获得受控访问权限, 并继续受到内部 ACL 策略约束; 校园内部 ACL 能够保证办公楼等授权区域正常访问财务处, 而宿舍区至财务处的被正确拦截。ICMP Flood 防护机制将异常报文成功率由 100% 降低至 12.5%; 端口扫描检测模块能够在达到阈值后自动触发封禁, 并通过 SQLite 审计模块对相关事件进行记录, 为安全事件追踪与溯源提供支持。

但是, 项目仍存在若干可改进之处。在当前实验设计中, 宿舍区等低优先级区域在 HTB 策略激活后出现了较高的 UDP 丢包率, 未来可通过引入动态带宽借用策略, 在不影响财务处优先保障的前提下为低优先级区域提供更平滑的带宽供给。其次, 当前端口扫描检测和封禁逻辑运行于用户态 Python 进程, 在高速扫描场景下存在一定的检测延迟, 后续可考虑将状态追踪逻辑迁移至内核态实现, 以提升检测响应速度。在实验参数设置方面, 当前实验中服务器出口链路的瓶颈带宽固定为 20 Mbps, 未来可在不同瓶颈条件下重复实验, 以验证各策略模块在不同负载强度下的鲁棒性。此外, 当前负载均衡采用 Round Robin 算法仅依据请求到达顺序进行服务器选择, 未考虑流量规模、持续时间及历史负载状态。当不同业务流的带宽需求存在显著差异时, 仍可能出现负载倾斜现象。未来可引入 Weighted Round Robin、Least Connection、Least Load 等感知流量特征的动态调度算法, 提高资源利用率与负载均衡效果。

5.4 对课程项目的建议

本次课程项目的实践过程中, 对 Mininet 在网络仿真中的作用有了更直观的理解。相比于课堂中的理论描述, QoS、负载均衡以及 ACL 等机制在实际部署后, 其运行过程和相互影响变得更加具体和可观察, 例如链路瓶颈位置、规则匹配顺序以及流量调度效果均会直接影响实验结果。这种“搭建—验证—调整”的过程使得对 Linux 网络子系统和 TCP/IP 协议栈运行机制的理解更加深入。

在后续实验设计中, 可以进一步扩展不同攻击场景与更复杂的网络拓扑, 以增强实验的对抗性与可扩展性。同时, 若引入更多真实业务流模型, 例如混合长短流或多级优先级业务, 将有助于更全面地评估 QoS 与负载均衡策略在复杂网络环境下的表现。

六、参考文献

- [1] R. Sherwood, G. Gibb, K.-K. Yap, G. Appenzeller, M. Casado, N. McKeown, and G. Parulkar, “FlowVisor: A network virtualization layer,” OpenFlow Switch Consortium, Tech. Rep. OPENFLOW-TR-2009-1, Oct. 2009.
- [2] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: Rapid prototyping for software-defined networks,” in Proc. ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets), Monterey, CA, USA, Oct. 2010, pp. 19–24. doi: 10.1145/1868447.1868466
- [3] S. Floyd and V. Jacobson, “Link-sharing and resource management models for packet networks,” IEEE/ACM Transactions on Networking, vol. 3, no. 4, pp. 365–386, Aug. 1995. doi: 10.1109/90.413235
- [4] M. Shreedhar and G. Varghese, “Efficient fair queuing using deficit round-robin,” IEEE/ACM Transactions on Networking, vol. 4, no. 3, pp. 375–385, Jun. 1996. doi: 10.1109/90.506411
- [5] M. Devera, “HTB Linux queuing discipline manual,” Linux Advanced Routing & Traffic Control HOWTO, 2002.
- [6] L. Kleinrock, Queueing Systems, Volume 2: Computer Applications. New York, NY, USA: Wiley, 1976. ISBN: 978-0471491118
- [7] M. Mitzenmacher, “The power of two choices in randomized load balancing,” IEEE Transactions on Parallel and Distributed Systems, vol. 12, no. 10, pp. 1094–1104, Oct. 2001. doi: 10.1109/71.963420
- [8] W. R. Cheswick and S. M. Bellovin, Firewalls and Internet Security: Repelling the Wily Hacker. Reading, MA, USA: Addison-Wesley, 1994.
- [9] J. Garcia-Alfaro, F. Cuppens, N. Cuppens-Boulahia, S. Martinez, and J. Cabot, “Management of stateful firewall misconfiguration,” Computers & Security, vol. 39, pp. 64–85, 2013. doi: 10.1016/j.cose.2013.01.004

- [10] M. Roesch, “Snort: Lightweight intrusion detection for networks,” in Proc. 13th USENIX Conference on System Administration (LISA), Seattle, WA, USA, Nov. 1999, pp. 229–238. Available: <https://dl.acm.org/doi/10.5555/1039834.1039864>
- [11] D. E. Denning, “An intrusion-detection model,” IEEE Transactions on Software Engineering, vol. SE-13, no. 2, pp. 222–232, Feb. 1987. doi: 10.1109/TSE.1987.232894
- [12] K. Ilgun, R. A. Kemmerer, and P. A. Porras, “State transition analysis: A rule-based intrusion detection approach,” IEEE Transactions on Software Engineering, vol. 21, no. 3, pp. 181–199, Mar. 1995. doi: 10.1109/32.385298
- [13] S. Mirkovic and P. Reiher, “A taxonomy of DDoS attack and defense mechanisms,” ACM SIGCOMM Computer Communication Review, vol. 34, no. 2, pp. 39–53, Apr. 2004. doi: 10.1145/997150.997156
- [14] U. Shankar and V. Paxson, “Active mapping: Resisting NIDS evasion without altering traffic,” in Proc. IEEE Symposium on Security and Privacy (SP’03), Oakland, CA, USA, May 2003, pp. 44–61. doi: 10.1109/SECPRI.2003.1199326